

User as Engram: Internalizing Per-User Memory as Local Parametric Edits

Anonymous submission

Abstract

Personal memory requires both mutable user facts and reusable reasoning. Retrieval-augmented generation (RAG) keeps facts external but pays a search or tool hop and serializes retrieved text at query time; per-user LoRA removes that path but stores each user as a global model update. We propose **User as Engram**: encode each user fact as embedding values and write them directly into deterministically addressed rows of a pretrained Engram memory table. These surgical weight edits carry the user knowledge; changing them does not retrain the backbone or its shared reasoning capability. Against per-user LoRA, the design provides three benefits. At 100 facts it uses 88 KB rather than 14.2 MB ($161\times$ less state); closed-form writes and addressed deletion avoid per-user retraining; and it matches direct recall while improving mean indirect reasoning by $5.6\times$, with its largest LOCOMO gains on multi-hop questions. Writes are inspectable and cause about $33,000\times$ less held-out loss increase. Against RAG, Engram offers an in-network lookup opportunity at every prefill and decode position, requiring neither external retrieval nor serialized memory tokens. On our controlled split, same-base RAG top-3 performs one search, adds about 29 memory tokens, and reaches 39.0% indirect accuracy versus Engram’s 44.5%. A $2.5\times$ larger Qwen-RAG top-3 leads at 34 facts, ties at 100, and falls to 30.0% versus 44.5% at 1,000 facts as retrieval recall degrades. Engram thus targets compact, frequently accessed user state, while RAG remains complementary for provenance and open-domain evidence.

Introduction

Personalization requires a model to acquire mutable user facts without turning every recall into an external retrieval or every update into global model adaptation. Retrieval-augmented generation (RAG) keeps facts editable, but each recall searches a store, serializes retrieved text, and pays the resulting context and prefill cost (Lewis et al. 2020; Gao et al. 2023); an agentic implementation also generates and executes a tool call. Per-user LoRA (Hu et al. 2022) removes that external path, but represents a user’s records as a model-wide low-rank update. It requires per-user optimization, provides

no explicit address for replacing or deleting one fact, consumes 14.2 MB at rank 64 even for a small record set, and can perturb unrelated behavior. Neither approach provides an internal fact store whose individual writes are explicit and structurally local.

We propose *User as Engram*: write facts directly into the model as surgical edits to an Engram memory table (Cheng et al. 2026). For each fact, deterministic hashes of its trigger suffixes select a small set of memory rows; we encode the fact as value embeddings and override only those addressed parameters. The rows are part of the pretrained model, and their gated values enter the residual stream during the same forward pass as ordinary Transformer computation. Every changed address is therefore enumerable, while the backbone and all unaddressed rows remain bit-identical. Unlike LoRA, the fact is not distributed through a model-wide update; unlike RAG, it is not fetched and serialized as text at query time.

The Engram embeddings—not a per-user adapter—carry the user-specific knowledge. Engram pretraining establishes the address-and-gate pathway that indexes these rows, while the model’s pretrained or post-trained general capability determines how their values are used. In our experiments, one shared LoRA strengthens this general reasoning capability, but it is only a complementary reader: it contains no test-user facts and remains fixed when an Engram fact is inserted, replaced, or deleted. Changing user knowledge thus changes only the addressed Engram rows and never requires retraining the backbone or reasoning skill.

This representation makes internal memory directly manageable. A 100-fact override map is 88 KB, $161\times$ smaller than a rank-64 per-user LoRA; closed-form insertion costs <1 ms, addressed deletion requires no gradient steps, and optional row-only refinement can repair interactions among facts. At inference, suffix N -grams can activate these embeddings at every prefill or decode position without semantic search, a tool round trip, or retrieved context tokens. Across three seeds, Engram facts read by the shared capability reach 41.2% indirect accuracy versus 7.3% for per-user LoRA, and their largest LOCOMO gains occur on multi-hop questions. Figure 4 summarizes the storage, lifecycle, and use benefits.

Figure 1 illustrates the roles. A user’s doctor is stored in the highlighted Engram embeddings; if that doctor changes, only those rows are replaced. Inferring that a cardiologist is

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

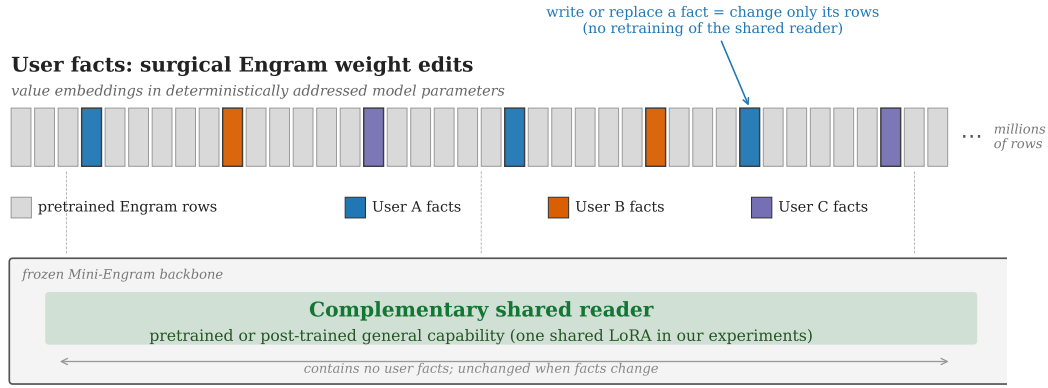


Figure 1: User facts reside in surgical Engram-row weight edits. A shared pretrained or post-trained capability reads them and remains fixed when the facts are inserted, replaced, or deleted.

relevant to a heart check-up is a general capability that may be learned during pretraining or post-training and reused unchanged. The reader enables use of the memory, but the fact itself resides entirely in the surgical Engram edit.

Contributions. We make three contributions. First, we introduce a protocol that writes a new user fact as embedding-valued, deterministically addressed edits to an Engram-pretrained model’s memory parameters. Second, we develop closed-form, per-fact, and joint writes together with addressed deletion, composable per-user maps, and mechanistic audits of the exact edit footprint. Third, we compare this internal memory with per-user LoRA and RAG across storage, update cost, locality, reasoning, context tokens, and knowledge-base scaling, and test whether a user-independent capability can operate on facts stored only in Engram rows. Engram uses $161\times$ less per-user state than LoRA and no external memory hop, while the $2.5\times$ larger Qwen-RAG baseline falls below it beyond 100 facts (Figures 4, 7, and 8).

Background and Design Requirements

Personal-memory mechanisms span three families. Prompt memories and retrieval stores keep content editable, but consume context and select evidence at every turn (Lewis et al. 2020; Gao et al. 2023). Parameter-efficient tuning moves content into a shared function (Hu et al. 2022). Model editing targets particular associations, yet usually produces weight changes whose locality is empirical (Meng et al. 2022, 2023). Engram pretraining supplies a fourth substrate: explicit key-value rows in the residual stream (Cheng et al. 2026).

We evaluate personal-memory methods along six requirements that are easy to conflate. *Recall* asks whether the exact fact can be produced. *Use* asks whether it can participate in a novel inference. *Locality* asks whether the write leaves unrelated behavior unchanged. *Lifecycle* requires individual update and deletion. *Serving* requires many users to share one backbone. *Access* asks whether each recall needs search, a tool round trip, or extra context. A method that succeeds on recall alone can fail all five remaining requirements; this motivates reporting held-out loss, indirect questions, address-level audits, per-user storage, and query cost alongside accuracy.

write or replace a fact = change only its rows
(no retraining of the shared reader)

The term *Engram* here denotes the model’s pretrained hashed memory table, not an arbitrary cache added after training. Its gates have learned when table values should enter the residual stream. We exploit that learned interface but do not alter its addressing rule or backbone. Our primary contribution is the write and composition protocol that turns selected Engram parameters into an editable store for new user facts. The shared adapter is not part of that store; it is a controlled way to test whether a user-independent reasoning capability can consume the writes, and it remains unchanged when facts change.

Why Facts Should Not Be Per-User LoRA

Weights are appropriate for shared abstractions but problematic for mutable, tenant-specific records. We isolate this distinction using Mini-Engram-d20 and the same 20 users. On the canonical seed, a rank-64 LoRA trained on one fact per user raises held-out loss by 1.784 and worsens 17/20 users, whereas addressed Engram rows change loss by only 0.00005 and worsen none. Across three seeds the means are $+1.698$ versus $+0.000052$ ($\sim 33,000\times$), and 49/60 versus 0/60 users worsen.

The distinction is architectural, not merely an optimizer failure. A LoRA must move shared projections to raise the desired answer probability, so its update can fire on unrelated inputs. A keyed memory write is inert unless its address is read. Strong instruction-tuned bases can conceal the behavioral damage by already solving a question, but cannot remove the underlying loss increase.

Across Qwen2.5-3B/7B (Qwen Team 2025), Llama-3.1-8B (Grattafiori et al. 2024), and Mistral-7B (Jiang et al. 2023), only 0–20% of users score worse after personalization, and mean indirect accuracy changes by $+0.088$ to $+0.217$; on the base LM, 85% score worse and the mean is -0.133 . Instruction tuning can therefore mask the task-level effect of a global LoRA update, but the update remains nonlocal. Each per-user LoRA is still an alternative global model state, whereas sparse override maps share a backbone.

User as Engram

An Engram layer hashes token-suffix N -grams to memory rows, gates their values, and adds them to the residual stream.

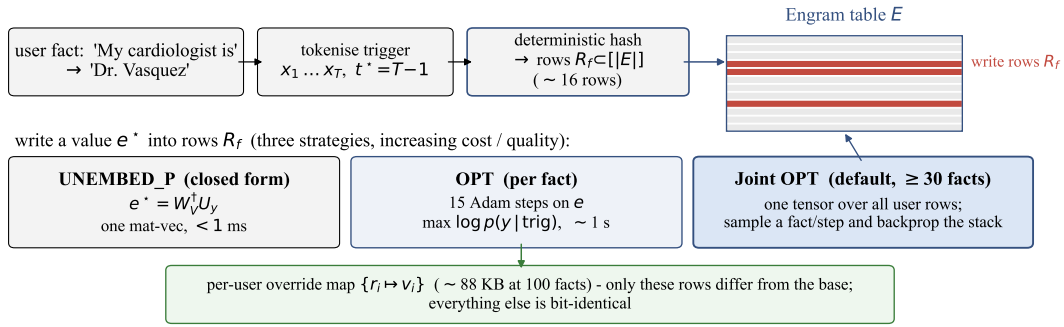


Figure 2: Surgical insertion into deterministically addressed Engram rows.

To write a fact, we put its answer where its trigger is read: deterministic hashes map the trigger’s suffix to a sparse address set R_f (about 16 rows for $N = 3, K = 8$). The backbone and every row outside R_f remain fixed. We evaluate three ways to choose the values written to R_f . A closed-form pseudo-inverse write (*UNEMBED_P*) uses the Engram value projection’s pseudo-inverse to align the row contribution with the answer token’s unembedding direction; it costs <1 ms and requires no gradient steps. Per-fact gradient optimization (*OPT*) starts from that value and minimizes answer loss by updating only R_f for each arriving fact. Joint optimization (*Joint OPT*) instead samples across a user’s facts while fitting the union of their addressed rows, so earlier and co-active facts remain compatible. All three strategies have the same editable parameter set; they differ only in write compute and recall. A 100-fact override map occupies about 88 KB. Figure 2 summarizes the write process.

The edit is a glass box. Figure 3 contrasts its footprint with the matched LoRA. For an inserted fact we can enumerate the trigger hashes, verify that only R_f changed, inspect the learned gates, measure the residual contribution at each layer, and observe the answer token rise in the logit lens. In the matched diagnostic, Engram’s residual delta is exactly zero at every non-trigger position and every pre-Engram layer; the LoRA delta is nonzero at every position and layer and moves unrelated text by a mean residual norm of 107. Locality therefore follows from the data structure rather than from a regularizer.

Keys, correctness, and deletion. Personal questions often reproduce an entity or relation phrase even when surrounding wording changes. Multiple suffix orders and hashes provide redundant triggers; inserting paraphrased question forms expands coverage. This is not semantic search: a query with no overlapping trigger may miss. Because addresses are deterministic, we know the complete editable row set before optimization. Deleting a user or fact removes its override entries rather than reversing a gradient update; updating a fact replaces explicit entries. Closed-form writes need no training, while optional joint re-optimization can repair interference when facts share rows.

Mechanistic checks. We validate locality four ways: byte comparison leaves the backbone and unaddressed rows identical; non-trigger positions have zero residual delta; the trigger gate rises from 0.02 to 0.99; and the residual change

aligns with the value path at cosine 0.999. The same write placed early drops recall from near-perfect to about one quarter, confirming that late placement matters.

The comparison LoRA is optimized to the same recall target. Its residual delta appears at all positions and layers, including an unrelated “capital of France” control, because low-rank does not mean locally activated. Norm or KL regularization might reduce that footprint, but would not make the update address-conditioned or identify exactly which inputs can activate it. For Engram, the zero-delta region follows directly from the lookup key.

Each user owns an override map $\Delta_u : h \mapsto v$. At inference, lookups first consult that map and otherwise fall back to the shared table. Maps with disjoint keys commute, so personal, team, and organizational memories can be stacked without merging dense weight deltas. Hash collisions are explicit and can be detected. In the current implementation the last write wins; maps whose triggers overlap should therefore remain separate.

Experiments

No pretrained Mini-Engram checkpoints are publicly available. We therefore integrate Engram lookup layers into Karpathy’s nanochat GPT-2-style backbone (Karpathy 2026; Radford et al. 2019) and pretrain all four Mini-Engram checkpoints ourselves from scratch with a nanochat-style recipe. We evaluate them against per-user LoRA, in-context facts, and retrieval, matching backbone, tokenizer, generation, and facts unless noted. Metrics cover direct and paraphrase recall, held-out LM loss, per-user storage, and indirect questions that transform stored facts.

Controlled bases vary depth and width but share the lookup. We report accuracy and held-out next-token loss because strong models can mask contamination once answers saturate. Storage excludes the backbone and shared adapter, which are amortized across tenants.

Synthetic schemas control recall and reasoning; LO-COMO adds conversational memory. Direct questions request stored answers, whereas indirect questions apply held-out relations and cannot be solved by echoing triggers. We repeat key contamination and layering results over three seeds. Writes are offline; query cost includes lookup, search, and context, and storage means serialized trainable state rather than duplicated checkpoints.

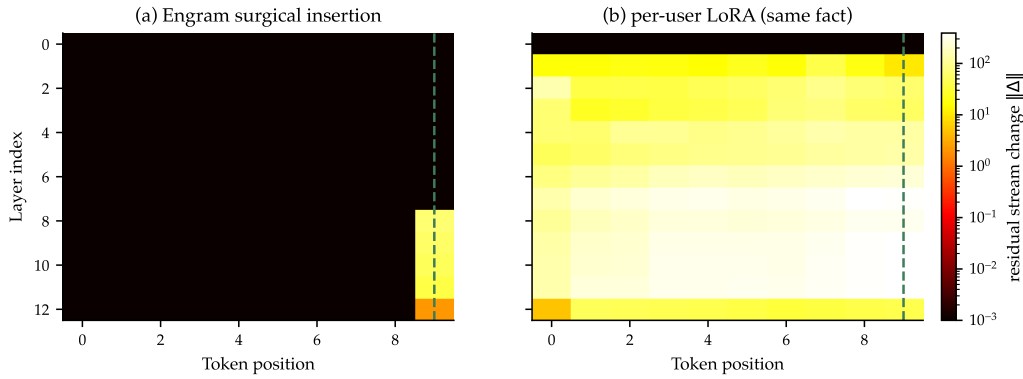


Figure 3: Residual changes from matched Engram and per-user LoRA edits.

Table 1: Mini-Engram backbones pretrained by us from scratch. All share the 51.2M-parameter memory table.

	d8 v2	d12 v2	d12@1280	d20@1536
depth $\times$ width	8×512	12×768	12×1280	20×1536
Engram layers	2, 5	2, 7	2, 7	2, 11
total parameters	178M	339M	625M	1.22B
ClimbMix tokens	0.50B	1.32B	3.34B	7.40B
validation bpb	.912	.827	.770	.730

Protocol and comparison controls

All Mini-Engrams use one lookup rule and a $50K \times 256$ (51.2M-parameter) table; Table 1 lists their scales. For pre-training, we use bf16 and the nanochat budget of 12 tokens per scaling parameter. Base, XL, and XXL corpora contain 100 USER + 100 ORG facts, 1,000 of each, and 3,132 trigger templates, respectively. At 100 facts, Joint OPT uses 2,000 Adam steps at rate 0.5. The three-seed test matches rank-64 LoRA (1,500 steps, 5×10^{-4} , $\alpha = 128$) against Joint OPT (1,500 steps, rate 0.5).

Metrics. Direct top-1 matches the first generated token to gold; `indirect_any` finds the gold span anywhere, and LOCOMO uses token-F1. Contamination is the change in held-out bits per byte (Δ bpb) on unrelated ClimbMix: 262K tokens for the canonical run and 524,288 for three-seed runs.

The shared rank-16 adapter trains on 10 users and tests on 20 disjoint users; contamination and high-density comparisons use rank-64 per-user Q/K/V LoRA. Engram sweeps vary insertion, density, model scale, triggers, and facts; per-rank, per-seed, and capacity tables remain available for supplementary review.

No-memory measures prior knowledge. For indirect reasoning, relations are learned from other users and test content enters only through the evaluated memory, requiring unseen-content access and transferable skill. Unrelated held-out ClimbMix makes Δ bpb measure collateral change, not fact recall.

Recall, storage, and lifecycle. A single edit is within eight points of the in-context ceiling. On Mini-Engram-d12, joint optimization handles interacting addressed rows at high density, with top-5 recall above 90% to roughly 300 facts/user. At 100 facts, its sparse override occupies 88 KB versus 14.2 MB for rank-64 LoRA. Unlike a model-shaped adapter, each row

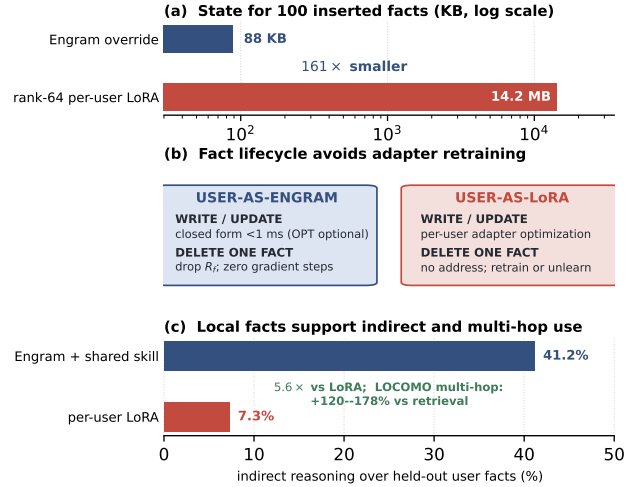


Figure 4: Engram-row benefits over per-user LoRA: (a) storage, (b) lifecycle, and (c) downstream use.

remains an explicit item that can be replaced or removed; UNEMBED_P provides a no-training write path, while OPT and Joint OPT trade write compute for recall. Figure 4 summarizes these storage, lifecycle, and downstream-use distinctions.

Memory-system and LOCOMO comparisons. Nearest-neighbor retrieval is strongest when queries repeat stored triggers. Our MEM0 and MemMachine baselines use only their retrieval recipes because facts are already structured; extraction and consolidation are out of scope (Chhikara et al. 2025; Wang et al. 2026). With all-MiniLM-L6-v2 (Wang et al. 2020; Reimers and Gurevych 2019), multi-trigger Engram gains 22 paraphrase points over the best retrieval baseline without context tokens.

Single-hop LOCOMO uses the first 80 non-adversarial questions from each of 10 conversations (~ 800) (Maharana et al. 2024). Retrieval stores gold evidence turns and Engram gold Q/A pairs: a controlled best-case comparison with a fixed Mini-Engram-d12 answering LM, not an end-to-end system. Across scales, Joint OPT leads on single-hop, multi-hop, and reasoning but trails on open-domain questions, where local facts cannot replace broad knowledge (Ta-

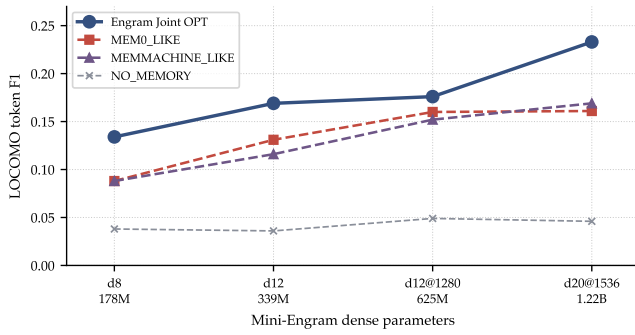


Figure 5: Locomo token-F1 across answering models.

ble 2; Figure 5). LongMemEval and PersonaMem-v2 provide complementary long-term and personalized memory evaluations (Wu et al. 2025; Jiang et al. 2025).

Joint OPT’s largest gains are multi-hop: token-F1 improves 120–178% over the better retrieval baseline, versus 10–58% on single-hop and reasoning. Open-domain performance falls 51–57%, favoring addressed personal memory plus retrieval for broad, changing knowledge.

Taken together with Figure 4’s controlled LoRA comparison, these results show that Engram facts can serve as operands for held-out transformations and multi-hop questions. Raw rows do not supply the reasoning algorithm: the key is that one shared skill can consume local facts, rather than relating both content and skill inside every user’s adapter.

The paraphrase result separates address coverage from answer quality. Retrieval is nearly perfect on repeated prefixes; more trigger variants improve Engram paraphrases without prompt tokens. Neither implies universal semantic generalization: a stored key must activate.

Multi-hop use. Across 63 two-fact chains, fact recall is 99.2% and direct second-fact matching is 90.6%, but true chaining falls to 12.9%: the store matches but does not compose. We therefore learn relations in a shared adapter on held-out users while fitting only test-user rows. The resulting layered system reaches 41.2% indirect accuracy across seeds, $5.6\times$ the per-user-LoRA baseline; the Locomo category results above independently show the largest gains on multi-hop questions.

Interpreting the comparisons. Across Table 3’s three seeds, matched recall isolates storage location, density tests coexistence, and layering tests whether locality sacrifices use. Recall need not cause global contamination, and reasoning can be shared; Locomo shows that addressed memory does not replace open-domain retrieval.

The $33,000\times$ loss ratio is a model-, fact-, and optimization-dependent stress test, not a constant. The robust distinction is qualitative: Engram can be inactive off-address, whereas LoRA changes the shared mapping. Likewise, 88 KB is the chosen 100-fact configuration, not a lower bound; width, precision, triggers, and collisions change it.

Negative results and boundary cases. UNEMBED_P is fast but underperforms when pretrained values misalign with

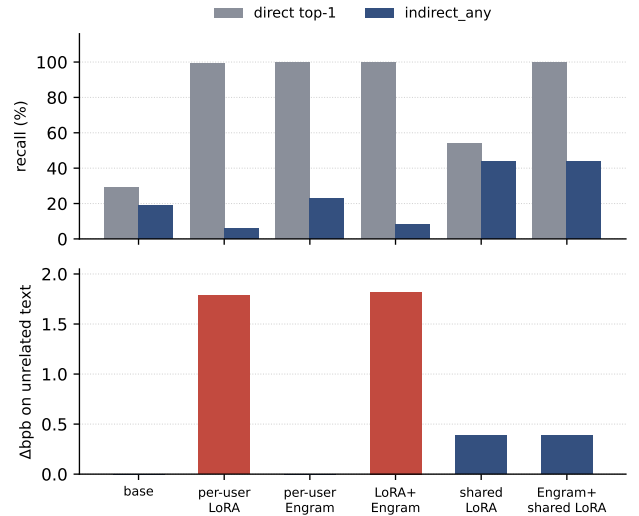


Figure 6: Direct and indirect recall and held-out Δ bpb across six conditions on Mini-Engram-d20 (seed S0, $n = 20$).

the answer. Independent fitting degrades under trigger collisions, and joint fitting repairs only part of the high-density loss. Free continuation is harder than short-span matching; unseen paraphrase triggers can fail. An arbitrary reasoning LoRA also fails unless it learns the same content interface, motivating layered training.

Reproducibility scope. The source retains insertion-strategy, rank, capacity, per-seed, pretraining, latency, judge-prompt, and alternative-architecture details for a planned supplement; the main paper retains the results needed for the causal chain.

Reasoning over Engram-Stored Facts

We use a rank-16 shared LoRA as a controlled reader of Engram-stored facts, not as the user-memory store. We train it on 10 users with each indirect question’s needed facts in context, freeze it, then test 20 disjoint users whose facts arrive only through Engram rows. Replacing a test-user fact changes only that user’s row map. Six conditions cross content placement (none, per-user LoRA, or Engram) with shared skill while holding direct recall nearly constant. Across three seeds, layered Engram reaches 100% versus 98.8% direct recall and 41.2% versus 7.3% indirect accuracy for per-user LoRA.

The layered system pays $+0.385$ Δ bpb once for the shared adapter; each user’s rows add only ≈ 0.00005 , versus $+1.698$ for a separate per-user LoRA. The adapter learns a reusable transformation while local rows supply user-specific arguments (Figure 6).

The layered-minus-LoRA indirect gain has paired-bootstrap 95% CI $[+31, +37]$ percentage points.

The factorial design attributes the gain: Engram without shared skill tests recall, shared skill without content tests leakage, and naive LoRA+Engram tests arbitrary stacking. Only intended layering gives local content, transferable reasoning, and low held-out loss; train and test users are disjoint.

Table 2: LOCOMO token-F1 by category and model. Joint OPT is compared with the stronger retrieval baseline in each cell.

category	answering LM	MEM0	MemMachine	Engram Joint OPT	lead vs. best retr.
single-hop	d12 v2	0.131	0.116	0.169	+29%
	d12@1280	0.160	0.152	0.176	+10%
	d20	0.161	0.169	0.233	+38%
multi-hop	d12 v2	0.092	0.076	0.243	+165%
	d12@1280	0.115	0.104	0.253	+120%
	d20	0.102	0.108	0.299	+178%
reasoning	d12 v2	0.117	0.113	0.183	+56%
	d12@1280	0.123	0.136	0.203	+49%
	d20	0.135	0.168	0.266	+58%
open-domain	d12 v2	0.247	0.209	0.122	−51%
	d12@1280	0.341	0.327	0.146	−57%
	d20	0.314	0.338	0.159	−53%

Table 3: Three-seed robustness on Mini-Engram-d20 ($n = 20$ users per seed).

Quantity	S0	S1	S2	Mean (range)
per-user LoRA Δ bp	1.784	1.754	1.557	1.698 (1.56–1.78)
per-user Engram Δ bp	.000052	.000049	.000054	.000052
layered total Δ bp	.386	.388	.379	.385 (.38–.39)
layered indirect_any	44.5%	44.2%	34.8%	41.2% (34.8–44.5)
per-user LoRA indirect_any	6.0%	8.8%	7.2%	7.3% (6.0–8.8)
worse than base (layered / LoRA)	0/20 / 17/20	0/20 / 16/20	0/20 / 16/20	0/60 / 49/60

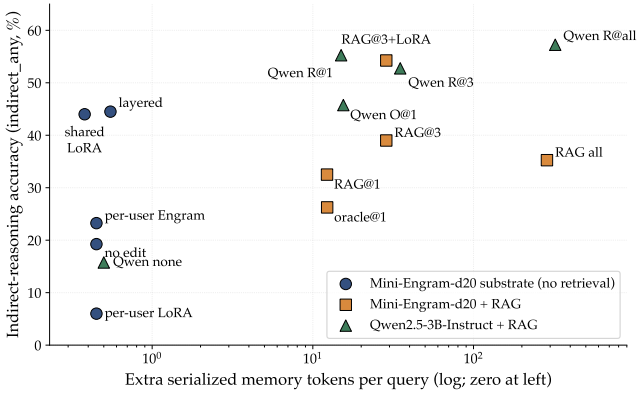


Figure 7: Engram versus RAG: indirect accuracy against extra serialized memory tokens per query; RAG points each perform one search.

Cross-schema transfer. The headline comparison uses disjoint users within one schema family. To test beyond those templates, we train on first-person personal facts and evaluate on third-person medical records. Layered indirect accuracy falls from 44% to 31% (30% relative), while per-user LoRA falls from 6% to 4%; the advantage persists ($7.4\times$ to $7.6\times$). Locality is unchanged: Δ bp stays $+0.386$ because personal rows activate only at addressed triggers.

Figure 7 compares the memory-access paths on the 34-fact split. Layered Engram reaches 44.5% indirect accuracy with no retrieved fact text or external memory call. RAG top-3 on the same answering model performs one search, serializes about 29 additional memory tokens, and reaches

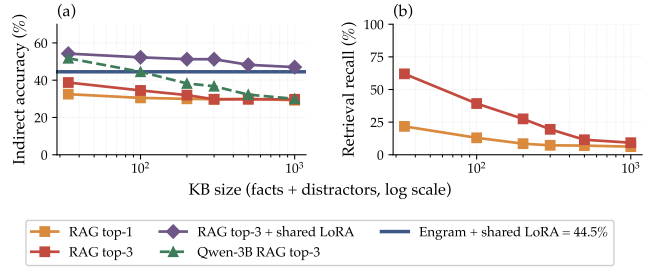


Figure 8: Engram versus RAG as the knowledge base grows: (a) indirect accuracy and (b) shared retriever recall. Qwen-3B uses a $2.5\times$ larger answering model.

39.0%; including all facts raises the memory block to 287 tokens and lowers accuracy to 35.3%. Combining RAG top-3 with the shared reasoning adapter instead reaches 54.3%, showing that retrieval and reusable skill are complementary. The $2.5\times$ larger Qwen2.5-3B-Instruct RAG baseline (Qwen Team 2025) reaches 51.8% with top-3 retrieval and 35 added memory tokens.

The distinction is access granularity. RAG searches once and passes a small set of facts through the prompt; an agentic pipeline additionally generates and executes the tool call, and sequential recalls repeat that external hop. Engram instead computes suffix addresses at every prefill and decode position and injects selected values inside the forward pass. It therefore supplies a token-granular memory channel without retrieved context, while retaining RAG as the better complement for rapidly changing corpora, provenance, and open-domain evidence.

To isolate knowledge-base size, we add held-out schema-family distractors to each user’s 34 facts, reaching $N \in \{34, 100, 200, 300, 500, 1000\}$ with the same 20 users and probes. The Mini-Engram RAG conditions use the same answering backbone as Engram; Qwen-RAG uses the larger instruction-tuned model. RAG top- k places the k highest-ranked facts in context, and all RAG conditions share the same retriever. Its top-3 recall falls from 62.0% at 34 facts to 9.3% at 1,000. Qwen-RAG top-3 consequently falls from 51.8% to 30.0%: it ties Engram’s constant 44.5% at 100 facts and trails thereafter. Same-base RAG top-3 falls from 38.8% to 29.8%. RAG plus the shared adapter remains slightly ahead at 1,000 facts (47.0%), again favoring a hybrid when retrieval’s provenance is needed (Figure 8).

Multi-Tenant Serving

A request resolves a sparse user table, saves and overrides affected base rows, runs one forward pass, then restores them. During that pass, each token’s suffix can fetch a gated memory value directly into the residual stream; there is no separate retrieval service or memory text to prefill. User switches change neither dense weights nor the shared adapter. Disjoint personal, team, and organizational scopes can layer; overlapping triggers are last-write-wins and should remain separate. Unmeasured batched serving would require one gather per Engram layer.

At one million users, 88 KB each is roughly 88–100 GB depending on accounting, plus one 11.8 MB shared rank-16 adapter; 14.2 MB per-user LoRAs require about 14.2 TB. Omitting allocator and index overheads, sparse content scales with stored facts while each LoRA scales with model dimensions.

S-LoRA and Punica (Sheng et al. 2024; Chen et al. 2024) batch dense adapters, but each tenant retains a model-shaped delta; overrides are indexed reads in the existing layer. Production requires untested caching, authorization, encryption, collision telemetry, and rate limits. The prototype demonstrates composition and scaling, not security.

The prototype uses sequential row swaps, not an optimized batched kernel. On one idle Blackwell GPU, d12@1280 processed 232 requests/s with 30 users and 50 facts/user (4.2 ms p50 latency), and 226 requests/s with 100 users and 100 facts/user (4.4 ms p50). These measurements are consistent with per-request work depending on the active override rather than the registered-user population, but they are not a controlled user-count scaling study because the fact count also changes. Absolute latency depends on cache residency, hardware, and the serving implementation.

Discussion, Limitations, and Related Work

Limitations. Within-user capacity remains limited: at 1,000 facts Joint OPT reaches 35% top-1 despite $<1\%$ occupancy, indicating forward-pass interference rather than collisions. Recall depends on trigger and paraphrase coverage, and insertion is slower than appending a document. LOCOMO mixes memory with base knowledge; storage and latency are implementation-dependent. We have not tested batched serving, adversarial exfiltration, or production access

controls. Most controlled tests use templated, short-answer facts; LOCOMO adds conversational evidence, but does not establish long-form generation, temporally conflicting memories, or semantic access without a matching trigger.

Our work connects per-user adapters (Houlsby et al. 2019; Hu et al. 2022; Zhang et al. 2024; Su et al. 2025; Tan, Liu, and Jiang 2024; Zhuang et al. 2024), retrieval-augmented generation (Lewis et al. 2020; Gao et al. 2023; Borgeaud et al. 2022), model editing (Meng et al. 2022, 2023; Mitchell et al. 2022a,b; Wang et al. 2024), and external or semi-parametric memory (Lample et al. 2019; Packer et al. 2023; Cheng et al. 2026). Unlike external access or global weight changes, User as Engram stores explicit, mutable records as surgical edits to addressed model parameters and consumes them in-network. This supports local, high-frequency, multi-tenant memory. The measured dense-adapter interference relates to catastrophic forgetting (McCloskey and Cohen 1989; Kirkpatrick et al. 2017). Compared with conventional model editing, the inserted fact need not already exist in dense weights, and locality follows from address-conditioned activation rather than a learned regularizer. Compared with per-user adapters, storage grows with the written rows rather than model dimensions. These structural advantages make updates inspectable and reversible, but constrain recall to the trigger contract.

Societal considerations. Addressability aids inspection and erasure but can expose records; output auditing and shared-skill fairness also remain open concerns.

Future work. Future recipes should test multi-fact training, random per-user hash offsets, and “recall, then reason” examples (Sun et al. 2023). Preliminary multi-fact finetuning sped convergence without raising the ceiling; teacher traces lowered accuracy when training and evaluation formats diverged.

Because our current pretraining reaches only 1.22B parameters, future work should scale the architecture and training recipe to substantially larger models. This will test whether its locality, recall, and multi-user efficiency persist at frontier-relevant scales.

Conclusion

User as Engram writes user facts directly into a model as embedding-valued, surgical edits to deterministically addressed Engram rows. Only those parameters change, and their gated values are read in the ordinary forward pass. At 100 facts, it matches per-user LoRA recall with $161\times$ less state and about $33,000\times$ less held-out loss increase, while requiring neither RAG’s retrieval call nor serialized memory text. A shared reader improves indirect reasoning by $5.6\times$ over per-user LoRA but contains no user facts: knowledge remains in the Engram edits, so facts can be inserted, replaced, or deleted without retraining that capability. This gives model-internal personal memory the lifecycle of an editable store.

Use of generative AI tools. Generative AI tools provided editorial assistance; the authors reviewed and verified all text, claims, citations, figures, and reported results.

References

- Borgeaud, S.; Mensch, A.; Hoffmann, J.; et al. 2022. Improving Language Models by Retrieving from Trillions of Tokens. In *Proceedings of the International Conference on Machine Learning*.
- Chen, L.; Ye, Z.; Wu, Y.; et al. 2024. Punica: Multi-Tenant LoRA Serving. In *Proceedings of MLSys*.
- Cheng, X.; Zeng, W.; Dai, D.; Chen, Q.; Wang, B.; Xie, Z.; Huang, K.; Yu, X.; Hao, Z.; Li, Y.; Zhang, H.; Zhang, H.; Zhao, D.; and Liang, W. 2026. Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models. arXiv:2601.07372.
- Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413.
- Gao, Y.; et al. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.
- Grattafiori, A.; Dubey, A.; Jauhri, A.; et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783.
- Houlsby, N.; et al. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the International Conference on Machine Learning*.
- Hu, E.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; et al. 2023. Mistral 7B. arXiv:2310.06825.
- Jiang, B.; Yuan, Y.; Shen, M.; et al. 2025. PersonaMem-v2: Towards Personalized Intelligence via Learning Implicit User Personas and Agentic Memory. arXiv:2512.06688.
- Karpathy, A. 2026. nanochat: An Experimental Training Harness for LLMs.
- Kirkpatrick, J.; Pascanu, R.; Rabinowitz, N.; et al. 2017. Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences*, 114(13): 3521–3526.
- Lample, G.; Sablayrolles, A.; Ranzato, M.; Denoyer, L.; and Jégou, H. 2019. Large Memory Layers with Product Keys. In *Advances in Neural Information Processing Systems*.
- Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*.
- Maharana, A.; et al. 2024. LOCOMO: Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*.
- McCloskey, M.; and Cohen, N. J. 1989. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In *Psychology of Learning and Motivation*, volume 24, 109–165. Academic Press.
- Meng, K.; Bau, D.; Andonian, A.; and Belinkov, Y. 2022. Locating and Editing Factual Associations in GPT (ROME). In *Advances in Neural Information Processing Systems*.
- Meng, K.; et al. 2023. MEMIT: Mass-Editing Memory in a Transformer. In *International Conference on Learning Representations*.
- Mitchell, E.; Lin, C.; Bosselut, A.; Finn, C.; and Manning, C. D. 2022a. Fast Model Editing at Scale. In *International Conference on Learning Representations*.
- Mitchell, E.; Lin, C.; Bosselut, A.; Manning, C. D.; and Finn, C. 2022b. Memory-Based Model Editing at Scale. In *Proceedings of the International Conference on Machine Learning*.
- Packer, C.; Wooders, S.; Lin, K.; et al. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.
- Qwen Team. 2025. Qwen2.5 Technical Report. arXiv:2412.15115.
- Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; and Sutskever, I. 2019. Language Models Are Unsupervised Multitask Learners (GPT-2). Technical report, OpenAI.
- Reimers, N.; and Gurevych, I. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*.
- Sheng, Y.; Cao, S.; Li, D.; et al. 2024. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. arXiv:2311.03285.
- Su, W.; et al. 2025. Parametric Retrieval-Augmented Generation (PRAG). arXiv:2501.15915.
- Sun, Z.; et al. 2023. Recitation-Augmented Language Models. In *International Conference on Learning Representations*.
- Tan, Z.; Liu, Q.; and Jiang, M. 2024. Democratizing Large Language Models via Personalized Parameter-Efficient Fine-Tuning (OPPU). In *Proceedings of EMNLP*.
- Wang, P.; Li, Z.; Zhang, N.; et al. 2024. WISE: Rethinking the Knowledge Memory for Lifelong Model Editing of Large Language Models. In *Advances in Neural Information Processing Systems*.
- Wang, S.; Yu, E.; Love, O.; Zhang, T.; Wong, T.; Scargall, S.; and Fan, C. 2026. MemMachine: A Ground-Truth-Preserving Memory System for Personalized AI Agents. arXiv:2604.04853.
- Wang, W.; Wei, F.; Dong, L.; Bao, H.; Yang, N.; and Zhou, M. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*.
- Wu, D.; Wang, H.; Yu, W.; Zhang, Y.; Chang, K.-W.; and Yu, D. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations*.
- Zhang, Z.; Rossi, R. A.; Kveton, B.; et al. 2024. Personalization of Large Language Models: A Survey. arXiv:2411.00027.
- Zhuang, Y.; et al. 2024. HYDRA: Per-User Adapters for Personalised LLMs.